

simplifynext

ignite
simplifynext

AGENTIC AI HACKATHON 2026

 Fueling Tomorrow's Changemakers



Centre for
Future-ready Graduates



Session schedule

Day 01

§1	LLM foundations How LLM works, Request and Response Structure
§2	Agentic AI The loop, tools, planning, memory Prompt Engineering Zero Shot, Few Shot, Chain of Thoughts, RAG
§3	Bedrock InvokeModel, tool calling, the ReAct loop

Day 02

§4	LangGraph State, nodes, edges, routing
§5	DeepAgents and the Claude Agent SDK Deep Agents Sub-agents,
§6	Agent Deployment using Bedrock AgentCore Configure, deploy, invoke, tear down

There will be Hands-on lab after every session.

§4 · LANGGRAPH

LangGraph: building the stateful graph.

State → Nodes → Edges → Tool Loop → Compile



LangGraph: building stateful agents.

LangGraph is an orchestration framework for stateful AI agents. Each node updates shared state, and edges determine how execution flows through the graph.

State

the central data structure that acts as shared memory across the entire workflow

Edges

The connector that routes execution from one node to the next.

Node

The worker function that reads the State, does a specific task, and returns updates.

Runtime

The engine that manages the execution, runs nodes, applies updates, and prevents infinite loops.

The State: the shared memory of the graph.

State: The single source of truth across the graph. It is typically a typed dictionary (*TypedDict* or *Pydantic* model) that stores memory, conversation history, and intermediate calculations. Every step reads from and updates this shared state.

```
from typing import Annotated, List
from typing_extensions import TypedDict
from langgraph.graph.message import
add_messages

class State(TypedDict):
    messages: Annotated[List, add_messages]
    documents: Annotated[List[str], operator.add]
    retry_count: int
    is_done: bool
```

The Schema Structure (What fields do you need?)

Determine what data must persist across nodes.

- **Chat / Conversation:** messages: Annotated[List, add_messages]
- **Task Data:** Input queries, extracted entities, or search results (e.g., documents: List[str])
- **Control Flags:** Loop counters, status flags, or routing values (e.g., retry_count: int, is_approved: bool)

Heavy payloads such as uploaded files, full document parses, and intermediate datasets should be stored in the graph State. Prompts should carry only metadata or references, keeping token costs low and context focused.

The Node: worker that reads State, does a job, writes back.

In LangGraph, a **Node** is defined as a Python function (or runnable) that accepts the current **State** as its argument and returns a dictionary of **State updates**.

```
def fetch_docs_node(state: State):
    # 1. Read from state
    # 2. Do work (logic, API call, LLM invocation)
    # 3. Return updates to state

    query = state["messages"][-1].content

    docs = search(query)

    return {
        "documents": docs,
        "messages": [AIMessage(content="Documents
retrieved.")],
    }
```

Every Node follows the same three steps

- **READ** — pull what you need out of State ([-1] means the most recent message)
- **WORK** — call an API, run an LLM, do a calculation
- **RETURN** — hand back a dictionary of changes

Nodes return partial updates, not complete State.

- A Node returns only those keys whose values have changed
- LangGraph performs the merge into the existing State automatically.
- Returned keys must correspond exactly to the field names declared in the schema, as an unmatched key is discarded silently and the update is lost.

Edges: The Wiring

Edges define the control flow of your graph. They determine the execution order and routing between nodes.

```
# 5. Build and Compile the Graph
builder = StateGraph(State)

# Add Nodes
builder.add_node("chatbot", chatbot_node)
builder.add_node("tools", tool_node)

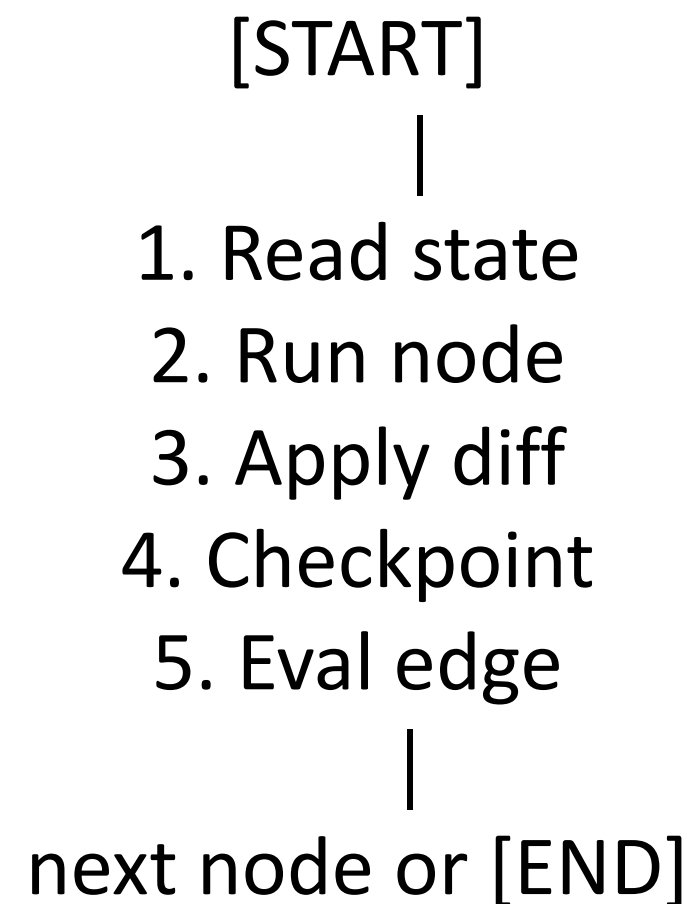
# Add Edges
builder.add_edge(START, "chatbot")
builder.add_conditional_edges("chatbot",
    should_continue, ["tools", END])
builder.add_edge("tools", "chatbot")
```

The 3 Types of Edges

- **Normal (Direct) Edges:** Unconditional transitions. As soon as Node A finishes, Node B always executes next.
- **Conditional Edges:** Dynamic routing. A routing function inspects the current State and decides which path to take (e.g., branching or looping).
- **Entry & Exit Edges:** Special edges that connect the entry point (START) and exit point (END) of the graph.

Runtime : The execution engine

Nodes and edges only describe what may happen. Once `builder.compile()` is called, the runtime takes over and actually walks the graph.

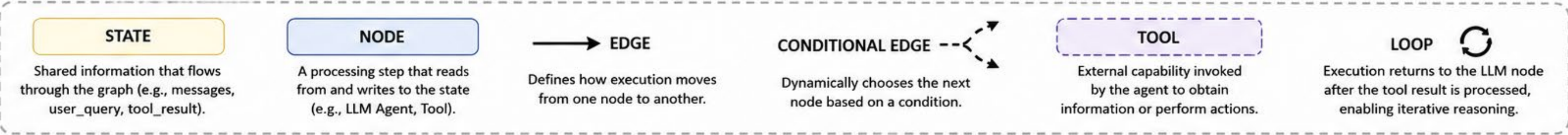
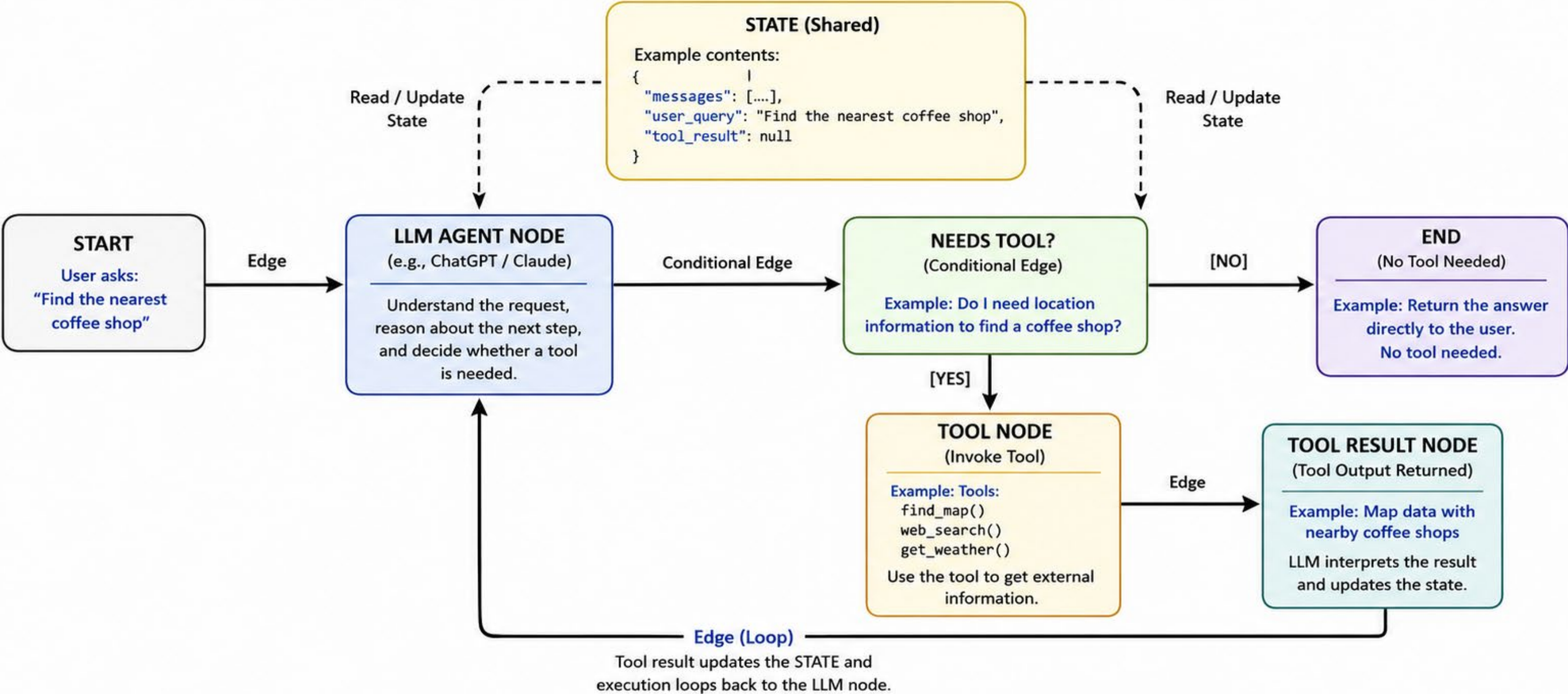


Three ways to run it

```
graph.invoke(state)
graph.ainvoke(state) # async
graph.stream(state) # per node
```

- **invoke** runs the whole graph and returns one thing: the final State. Nothing is visible until it finishes.
- **ainvoke** is the same call, awaited. Use it inside async code such as a web server.
- **stream** yields an update as each node finishes, so the user can watch progress instead of waiting.

State-Based LLM Agent Graph: Nodes, Edges, Conditional Edge, and Tool



LAB 04

Lang Graph

Basic Components

DURATION

15 minutes

FILE

Section_04_langgraph/



Lab : Lang Graph

Each file is a short script you can run on its own.

```
section_4_langgraph/  
└─ 00_langgraph_basics.py
```

FILE

WHAT THE SCRIPT DOES

00 langgraph_basics

Shows the four basic parts of LangGraph: a state that holds the messages, nodes that return only what they changed, edges that pick what runs next, and compile

§5 · DEEPAGENTS & AGENT SDK

From Graph Orchestration to Multi- Agent Architectures

The same concepts, packaged at a higher level of abstraction. Next, we'll explore Deep Agents, the Claude Agent SDK, and the trade-offs you make for that additional abstraction.



Choosing Agentic AI Frameworks

Now that we know the frameworks, how do we build an advanced agent capable of complex, multi-step tasks?

	Framework	Mental Model	Best For
	LangGraph	Flowchart (Nodes & Edges)	Strict step-by-step rules and workflows where you can't afford mistakes.
	CrewAI	Roleplay Team (e.g., Researcher + Writer)	Quick prototypes with specialized AI personas collaborating.
	OpenAI Agents SDK	Relay Race (Explicit Handoffs)	Clean agent-to-agent transfers if you build strictly on OpenAI models.
	Claude Agent SDK	Coding Assistant (Files & Terminal)	Autonomous coding, executing bash commands, and filesystem tasks.
	Google ADK	Corporate Tree (Boss $\rightarrow$ Sub-agents)	Hierarchical agent workflows built on Gemini and Vertex AI.
	Microsoft Agent Framework	Group Chat / Enterprise	Multi-agent collaboration for enterprise and .NET ecosystems (evolution of AutoGen).

There is no single "best" framework. Choose **LangGraph** when you need strict flowchart control, **CrewAI** for role-based teams, or a provider SDK (**OpenAI, Claude, Google, Microsoft**) if you are deeply tied to a specific cloud platform.

Deep Agents & Supervisor Pattern

The Core Idea: Instead of one agent trying to do everything in a single loop, a **Deep Agent** acts as a Project Manager that creates a plan and delegates tasks to focused **Sub-Agents**.

Supervisor = Strategy & Memory | Sub-Agents = Execution & Tools

1. The Supervisor (Project Manager)

- **Dynamic Planning:** Builds a todo list to break big goals into smaller steps.
- **Context Protection:** Prevents raw tool logs (like long web searches) from cluttering main memory.
- **Orchestration:** Assigns sub-tasks to specialists and updates progress as work completes.

2. Sub-Agents & MCP Tools (Specialists)

- **Specialized Sub-Agents:** Workers dedicated to specific tasks (e.g., Researcher, Coder, Writer).
- **Clean Handoffs:** Workers do heavy lifting in isolated loops and return short summaries.
- **MCP Server Integration:** Uses standardized protocols to plug into databases and tools effortlessly.

2. Create the Project Manager (Deep Agent)

```
from deepagents import create_deep_agent
```

```
deep_agent = create_deep_agent(  
    model="bedrock/claude-3-7-sonnet",  
    sub_agents=[researcher, coder],  
    enable_planning=True # Gives the supervisor a persistent TODO list )
```

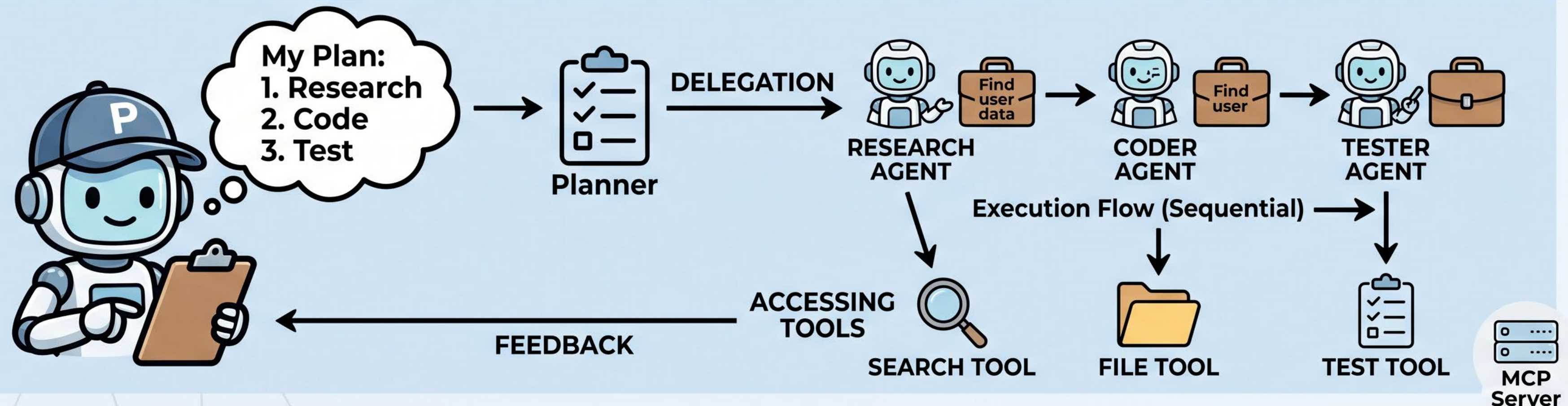

DEEP AGENT FLOW

The Project Manager vs. The Overwhelmed Agent (No Technical Acronyms!)

BEFORE: ONE SINGLE AGENT (Overwhelmed)



AFTER: THE DEEP AGENT (Organized Teamwork)



TAKEAWAY: A Deep Agent is just a Project Manager that makes a plan and gives small, simple jobs to a team, rather than trying to do everything alone.

Sub Agents : Specialized Worker

The Core Idea: Instead of one agent trying to do everything in a single loop, a **Deep Agent** acts as a Project Manager that creates a plan and delegates tasks to focused **Sub-Agents**.

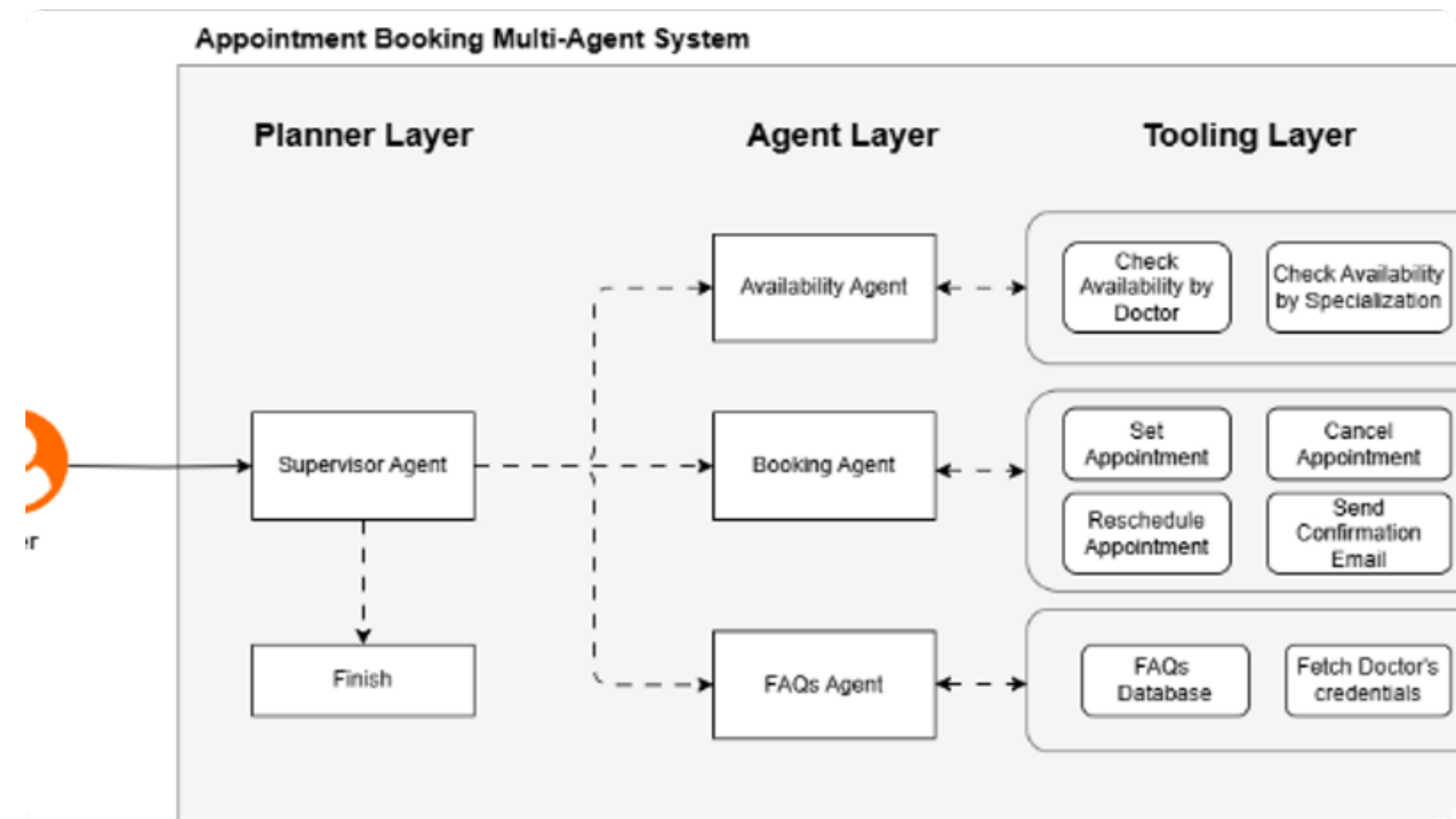
1. Dedicated Specialization

- Each sub-agent has narrow system prompts and specific tools (e.g., Coder gets file access; Researcher gets web search).

2. Context Isolation (No Memory Rot)

- Raw search results and trial-and-error tool logs stay inside the sub-agent loop—keeping the Supervisor prompt clean.

```
# Define a specialist sub-agent
researcher = {
  "name": "researcher",
  "tools": [lookup_tool],
  "system_prompt": "Return only concise facts."
}
# Attach to Supervisor
agent = create_deep_agent(model=chat_model(),
                          sub_agents=[researcher])
```



Tools as an MCP Server

A **tool** is an object, instrument, or piece of software designed to extend human capabilities and accomplish a specific task more efficiently.

Core Architecture Principles

- **Decoupled Microservice:** Tools run in their own process, completely separated from agent prompt engineering.
- **Dynamic Discovery:** The agent calls tools/list on startup to auto-detect tool signatures and inputs.
- **Universal JSON-RPC:** Any client (Claude, LangGraph, Deep Agents) can invoke tools via tools/call.

1. MCP TOOL SERVER (SERVER.PY)

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("DB-Server")

@mcp.tool()
def query_db(user_id: int) -> str:
    """Fetch user record from database."""
    return f"User {user_id}: Active, Tier=Pro"

if __name__ == "__main__":
    mcp.run()
```

2. DEEP AGENT HOST (AGENT.PY)

```
From _common import chat_model
from deepagents import create_deep_agent

agent = create_deep_agent(
    model=chat_model(),
    mcp_servers=["./Server.Py"],
    system_prompt="use MCP tools to assist users."
)

Res = agent.Invoke({"messages": [{"role": "user",
"content": "check user 42"}]})
```

LAB 05

Deep Agent

Basic Components

DURATION

15 minutes

FILE

Section_05_deepagents/



Total Lab : 6 scripts, one component each

Each file is a short script you can run on its own.

```
section_5_deepagents/  
├─ 01_abstraction_levels.py  
├─ 02_deepagent.py  
├─ 03_subagents.py  
└─ 04_agent_sdk.py
```

RUN IN ORDER 01 → 05

FILE	WHAT THE SCRIPT DOES
01 abstraction_levels	Asks the same stock question three times — hand-wired graph, create_agent, create_deep_agent — and prints what each cost in tokens and seconds. Rung 5 sends 11× the input.
02 deepagent	Gives one agent a lookup tool and a filesystem, then shows it writing a plan with the to-do tool before researching, and saving a briefing to a file.
03 subagents	Adds a reviewer sub-agent as a plain dict, and marks the moment the orchestrator hands work over with task.
04 agent_sdk	The Claude Agent SDK: tools bundled as an in-process MCP server, configured with ClaudeAgentOptions. Read-only without the claude CLI.

DeepAgents requires Claude. On Groq the harness uses llama-3.3-70b which fails (tool_use_failed).

§6 · DEPLOYING TO AGENTCORE

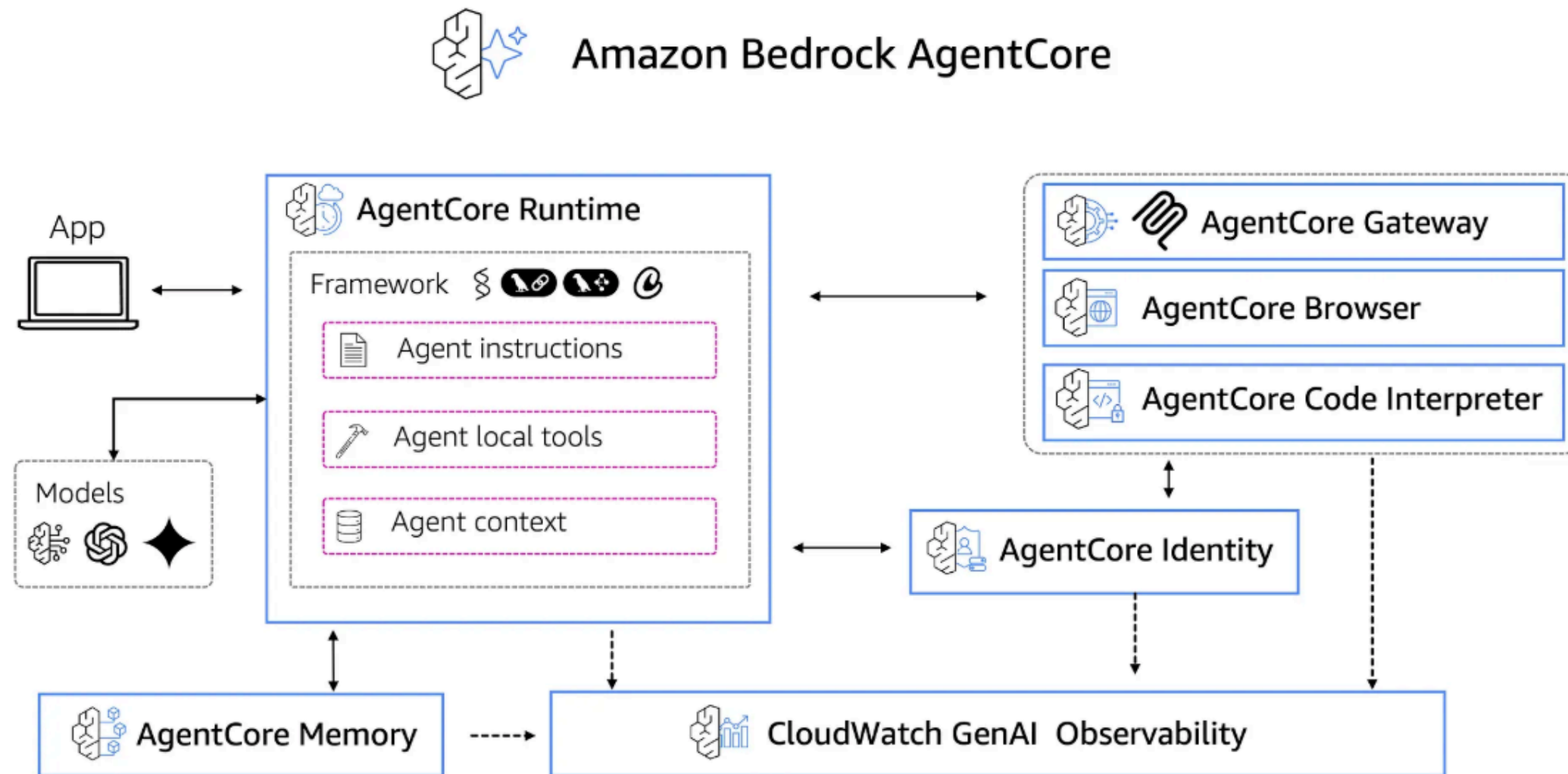
Deploy Agent to AWS Bedrock

Learn how to package, deploy, and expose your LangGraph agent as a production-ready HTTP service using Amazon Bedrock AgentCore.



Amazon Bedrock AgentCore

Managed pieces for running agents in production. They work with any framework and any model, and none of them require the others.



SOURCE : AWS BEDROCK ARCHITECTURE

Runtime

Serverless hosting for the agent, behind an HTTPS endpoint.

Memory

Managed short-term and long-term memory.

Identity

Who may call the agent, and what it may call for a user.

Gateway

Turns existing APIs and Lambda functions into MCP tools.

Browser · Code Interpreter

A managed browser and a sandboxed interpreter.

Observability

OpenTelemetry traces and spans, on CloudWatch.

The AgentCore Runtime

Bedrock AgentCore Runtime provides a secure, serverless, purpose-built hosting environment for AI agents. The SDK simplifies deployment: it wraps your agent in an HTTP server, then packages it as a container image, pushes it to ECR, and runs it serverless.

ONE DECORATOR

THE SDK SUPPLIES THE SERVER

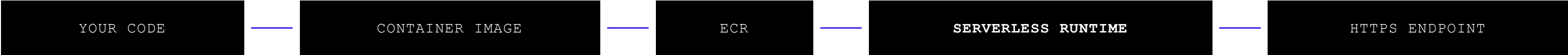
```
app = BedrockAgentCoreApp()

@app.entrypoint
def handler(payload):
    return agent(payload["prompt"])

if __name__ == "__main__":
    app.run() # port 8080
```

- POST /invocations
JSON in. JSON back, or SSE for streaming.
- GET /ping
Healthy, or Healthy/Busy while a background task runs.

BOTH ENDPOINTS ARE CREATED FOR YOU



AGENTCORE BUILDS AND PUSHES THE IMAGE · YOU DO NOT WRITE A DOCKERFILE

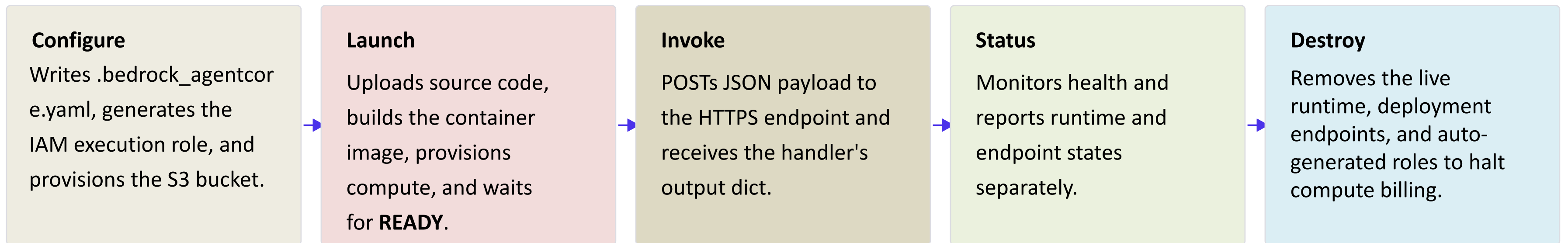
ONE MICROVM PER SESSION

Each session gets its own compute, memory and filesystem, provisioned on demand and torn down afterwards.

The decorator is the whole integration. Port, routes, content types and error handling are supplied, which is why the agent code from §4 needs no changes.

Deployment Lifecycle

Managing the End-to-End Cloud Footprint of an Agent Service



TEARDOWN IS NOT COMPLETE

destroy removes the runtime and its deployment resources. Shared infrastructure — S3 buckets, ECR repositories, CloudWatch log groups — can survive, depending on the configuration.

LAB 06

Agent Core

Basic Components

DURATION

15 minutes

FILE

Section_06_agentcore/



Total Lab : 6 scripts, one component each

Each file is a short script you can run on its own.

section_6_agentcore/

└─ agent/

| └─ agent.py

| └─ requirements.txt

└─ 01_run_local.py

└─ 02_deploy.py

└─ 03_tearardown.py

└─ 04_call_agent.py

RUN IN ORDER

01 → 02 → 04 → 03

FILE	WHAT THE SCRIPT DOES
01 run_local	Starts agent/agent.py on localhost:8080, POSTs one document, prints the JSON, stops the server. Free — no AWS resources.
02 deploy	Configure, launch, invoke. Creates the IAM role, uploads the source, waits for READY, then calls the live endpoint. Billable from here.
04 call_agent	Calls the deployed endpoint with plain boto3
03 teardown	Deletes the runtime and its resources.

agent/agent.py is the §4 graph plus four lines: the import, app = BedrockAgentCoreApp(), @app.entrypoint, and app.run(). Nothing about the graph changes.



AGENTIC AI HACKATHON 2026

Thank You

Fueling Tomorrow's Changemakers

www.simplifynext.com